

Steps to integrate ClearQuote SDK in a native IOS App

This document describes the steps needed to be followed for integration of ClearQuote SDK into any Mobile Application (Native IOS)

Prerequisites:

1. Supported destination is iPhone
2. Minimum iOS version supported = iOS 16
3. XCode version = 16.0+
4. Swift version = 5.x

Below are the detailed steps that need to be followed while integration SDK:

1. Add the ClearQuoteSDK

Please ignore steps **b** and **c** if cocoapods already setup in the source project

- a. In Xcode, open your project (xcproject OR xcworkspace)
- b. File → Add Package Dependencies...
- c. Add Repository URL: <https://github.com/clearquotetech/cq-ios-sdk.git>

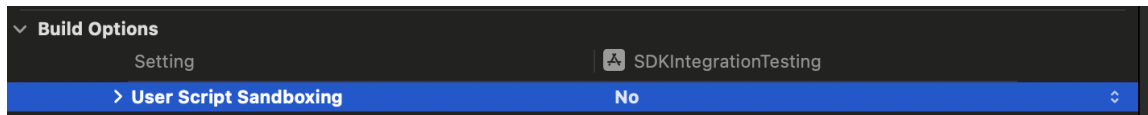
Swift

```
.package(  
    url: "https://github.com/clearquotetech/cq-ios-sdk.git",  
    .branch("main")  
)
```

- d. Add the ClearQuoteSDK product to your app target
- e. Also link these Apple frameworks on the app target for SDK runtime
 - i. CoreML
 - ii. MetalPerformanceShaders
 - iii. Accelerate

Note: From version 0.1.6 onwards, SDK will be released via SPM only

2. Make sure app sandboxing is off in build option of target app



3. Add the camera permission string in the source app's info.plist file

Key: Privacy - Camera Usage Description

Value: For capturing vehicle images

4. Add the location permission string in the source app's info.plist file

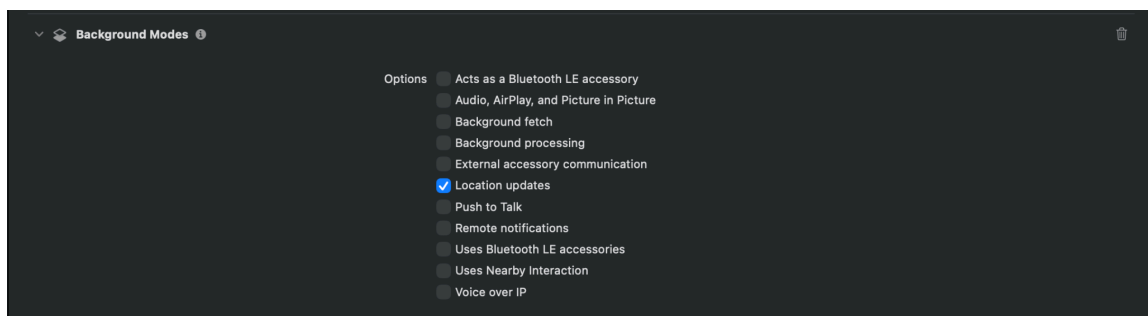
Key: Privacy - Location Always and When In Use Usage Description

Value: <your app name> needs your location while using the app to capture inspection locations and sync your inspections

Key: Privacy - Location When In Use Usage Description

Value: <your app name> needs your location while using the app to capture inspection locations and sync your inspections

5. Add Background modes support in the source app



6. Add TensorFlow Run Script

Add the following Run Script on the app target so the app can be uploaded to the App Store. Keep it after Embed Frameworks.

```
Swift
set -e || pipefail
```

```

VERSION="1.0.0"
MIN_OS="${IPHONEOS_DEPLOYMENT_TARGET}"

patch_plist() {
    plist="$1"
    if [ ! -f "${plist}" ]; then
        return 0
    fi

    if /usr/libexec/PlistBuddy -c "Print :CFBundleShortVersionString" "${plist}"
>/dev/null 2>&1; then
        /usr/libexec/PlistBuddy -c "Set :CFBundleShortVersionString ${VERSION}"
"${plist}"
    else
        /usr/libexec/PlistBuddy -c "Add :CFBundleShortVersionString string ${VERSION}"
"${plist}"
    fi

    if /usr/libexec/PlistBuddy -c "Print :MinimumOSVersion" "${plist}" >/dev/null
2>&1; then
        /usr/libexec/PlistBuddy -c "Set :MinimumOSVersion ${MIN_OS}" "${plist}"
    else
        /usr/libexec/PlistBuddy -c "Add :MinimumOSVersion string ${MIN_OS}" "${plist}"
    fi

    echo "Patched ${plist}"
}

# 1) Patch SourcePackages artifacts so future embeds already have valid keys.
DERIVED_DATA_ROOT="${BUILD_DIR%/Build/*}"
ARTIFACTS_ROOT="${DERIVED_DATA_ROOT}/SourcePackages/artifacts/tensorflowlitec"
if [ -d "${ARTIFACTS_ROOT}" ]; then
    find "${ARTIFACTS_ROOT}" -path "*/TensorFlowLiteC*.framework/Info.plist" | while
IFS= read -r plist; do
        patch_plist "${plist}"
    done
else
    echo "note: TensorFlowLiteC SPM artifacts not found at ${ARTIFACTS_ROOT}"
fi

# 2) Patch (and re-sign) frameworks already embedded in the app bundle.
FRAMEWORKS_DIR="${TARGET_BUILD_DIR}/${FRAMEWORKS_FOLDER_PATH}"
if [ -d "${FRAMEWORKS_DIR}" ]; then
    for name in TensorFlowLiteC TensorFlowLiteCMetal TensorFlowLiteCCoreML; do

```

```

fw="${FRAMEWORKS_DIR}/${name}.framework"
patch_plist "${fw}/Info.plist"
if [ -d "${fw}" ] && [ -n "${EXPANDED_CODE_SIGN_IDENTITY:-}" ] && [
"${EXPANDED_CODE_SIGN_IDENTITY}" != "-" ] && [ "${CODE_SIGNING_ALLOWED:-YES}" !=
"NO" ]; then
    /usr/bin/codesign --force --sign "${EXPANDED_CODE_SIGN_IDENTITY}"
--preserve-metadata=identifier,entitlements,flags --timestamp=none "${fw}"
fi
done
else
    echo "note: Frameworks directory not found at ${FRAMEWORKS_DIR}; skipping
embedded patch"
fi

```

7. Import the ClearQuoteSDK module

```

import ClearQuoteSDK

class SDKInitializationViewController: UIViewController {
    ...

    override func viewDidLoad() {
        ...
    }
}

```

8. Initialize the ClearQuote SDK

```

class SDKInitializationViewController: UIViewController {
    ...

    override func viewDidLoad() {
        ...

        // Initialize the ClearQuote SDK
        ClearQuote.shared.initSDK(
            baseVC: UIViewController,
            key: String,
            result: { isInitialized, code, message in
                if (isInitialized) {

```

```

        // Handle success
    } else {
        // Handle error
    }
}
)
}
}

```

Note

1. Please avoid initializing the SDK again if it is already initialized
2. For checking if sdk is already initialized use this public function, `ClearQuote.shared.isCQSDKInitialized()`
 - a. Returns true, if sdk is initialized
 - b. Returns false, if sdk is not initialized

Explanation for the parameters in `initSDK()`

1. `baseVC: UIViewController`
 - a. It can be any UI Viewcontroller in the source app which is responsible for initializing the ClearQuote SDK
2. `Key: String`
 - a. Please contact ClearQuote team to get the sdk key
3. `result: @escaping ( _ isInitialized: Bool, _ code: Int, _ message: String) -> Void`
 - a. This callback function is useful to handle sdk initialization success and error cases
 - b. `isInitialized` is a boolean variable
 - i. If true, sdk initialized successfully
 - ii. If false, sdk is not initialized
 - c. Possible codes and messages are explained further

Possible messages from ClearQuote SDK while initializing

1. Error messages
 - a. "SDK key is empty"
 - i. Please recheck if the SDK key is passed correctly or not
 - b. "Invalid user region"
 - i. Please recheck if the SDK key is passed correctly or not
 - c. "SDK Internal Error"
 - i. Contact ClearQuote team in this case
2. Success message

- a. "Success"
 - i. ClearQuote SDK is initialized successfully

Possible error codes from ClearQuote SDK while initializing

- 1. Error code
 - a. 101 - SDK key is empty
 - b. 100 - Invalid user region
 - c. 500 - SDK Internal Error
- 2. Success code
 - a. 200 - Success

9. Start Inspection

Before starting an inspection initializing the ClearQuoteSDK is important

```
import ClearQuoteSDK

class StartInspectionViewController: UIViewController {
    ...

    @IBAction private func onClickBtnStartInspection() {
        ClearQuote.shared.startInspection(
            baseVC: UIViewController,
            clearQuoteSdkDelegate: ClearQuoteSDKDelegate,
            clientAttrs: CQSDKClientAttrs(
                userName: String?,
                dealer: String?,
                dealerIdentifier: String?,
                client_unique_id: String?,
                organisationId: String?
            ),
            inputDetails: CQSDKInputDetails(
                customerDetails: CQSDKCustomerDetails(
                    name: String?,
                    email: String?,
                    dialCode: String?,
                    phoneNumber: String?
                ),
                vehicleDetails: CQSDKVehicleDetails(
                    regNumber: String?,
                    make: String?,
```


- a. Username of logged in user inside the source app
 - b. This value will be stored in quote details
- 5. clientAttrs.dealer: String? = nil
 - a. A valid dealer code in ClearQuote system
 - b. This value will be stored in quote details
- 6. clientAttrs.dealerIdentifier: String?
 - a. If source app want ClearQuote sdk to handle any unique user group id in source system to user group id in CQ system then source system's unique user group id can be passed here
 - b. This value will be stored in quote details
- 7. clientAttrs.organisationId: String?
 - a. Optional client-provided organisation id for the inspection
 - b. This value will be stored in quote details
- 8. inputDetails: CQSDKInputDetails? = nil
 - a. All values of inputDetails keys are optional with default value as nil
 - b. Basic details about inspection such as vehicle & customer details can be sent in this parameter, default value for this parameter and all the keys under this object is set as nil
- 9. inputDetails.customerDetails.name: String? = nil
 - a. Customer name
- 10. inputDetails.customerDetails.email: String? = nil
 - a. Customer email
- 11. inputDetails.customerDetails.dialCode: String? = nil
 - a. Customer phone number's dial (or country) code
- 12. inputDetails.customerDetails.phoneNumber: String? = nil
 - a. Phone number of a customer
- 13. vehicleDetails.regNumber: String? = nil
 - a. Registration number of a vehicle that source app user wants to inspect
 - b. If your account has a fleet list enabled then only sending vehicle Registration number is enough for starting an inspection, other details such make, model & bodystyle are not required
 - c. Please check with the CQ team to know if fleet list is enabled for the account given to you
- 14. vehicleDetails.make: String? = nil
 - a. Make of a vehicle that source app user wants to inspect
- 15. vehicleDetails.model: String? = nil
 - a. Model of a vehicle that source app user wants to inspect
- 16. vehicleDetails.bodyStyle: String? = nil

- a. Bodystyle of a vehicle that source app user wants to inspect
- 17. vehicleDetails.fuelType: String? = nil
 - a. Fuel type of a vehicle that source app user wants to inspect
- 18. vehicleDetails.variant: String? = nil
 - a. Variant of a vehicle that source app user wants to inspect
- 19. quoteData.inspectionType: String? = nil
 - a. Different types of inspection types can be passed for example Pre Trip, Post Trip to differentiate between inspections
 - b. This value will be stored in quote details
- 20. quoteData.fleetImageType: String? = nil
 - a. If your account has Check-In and Check-Out inspection comparison feature enabled then following are the only permissible values
 - i. "before" (Case-sensitive)
 - ii. "after" (Case-sensitive)
 - b. If nothing is passed then by default "before" will be considered
 - c. Please check with the CQ team to know if this feature is enabled for your account
- 21. userFlowParams.isOffline: Bool? = nil
 - a. If value is passed true, ClearQuote sdk will start inspection in offline mode
 - b. If value is passed false, ClearQuote sdk will start inspection in online mode (Default)
 - c. By default ClearQuote SDK will start the online inspection
 - d. Please check with the CQ team to know if this feature is enabled for your account
- 22. userFlowParams.skipInputPage: Bool? = nil
 - a. If value is passed true, ClearQuote sdk will start the inspection from camera page
 - b. If value is passed as false, ClearQuote sdk will start the inspection from input page (Default)
 - c. By default ClearQuote SDK will start the inspection from input page

Possible messages from ClearQuote SDK while starting an inspection

- 1. Error messages
 - a. In case of skip input
 - i. "Empty Registration Number is not allowed"
 - ii. "Special characters in Registration Number are not allowed"
 - iii. "Empty make is not allowed"

- iv. "Null make is not allowed"
 - v. "UnAcceptable make"
 - vi. "Empty model is not allowed"
 - vii. "Null model is not allowed"
 - viii. "UnAcceptable model"
 - ix. "Error in creating a quote"
 - x. "Error in fetching overlays"
- b. In case of inspection starting from input page
 - i. "CQ SDK is not initialized"
- c. In case of offline inspection is triggered but config data is not available due to some technical issue
 - i. "Sufficient data not available to create an offline quote"
- d. In case of offline quote creation is not allowed
 - i. "Not permitted to create offline quote"
- 2. Success message
 - a. "Success"

Possible error codes from ClearQuote SDK while starting an inspection

- 1. Error code - inspection not started
 - a. 500
- 2. Success code - inspection started successfully
 - a. 200

10. Get inspection completion status

- a. Create an extension of the source app's view controller that is responsible for handling the inspection status and confirm with the "ClearQuoteSDKDelegate" protocol
- b. Use "ClearQuoteSDKDelegate" protocol delegate to access inspection completion status as shown below

```
import ClearQuoteSDK
...

extension StartInspectionViewController: ClearQuoteSDKDelegate {
    func inspectionCompletionStatus(identifier: String, message: String, code: Int) {

    }
}
```

}

Explanation for the methods in ClearQuoteSDKDelegate

1. inspectionCompletionStatus(identifier: String, message: String, code: Int)
 - a. identifier: String
 - i. Value will be "ClearQuote"
 - b. message: String
 - i. Possible values are as follows
 1. "Success" - User submitted the inspection successfully
 2. "Failure" - User did not complete the flow
 - c. code: Int
 - i. Possible values are as follows
 1. 200 - "Success" - User submitted the inspection successfully
 2. 400 - "Failure" - User did not complete the flow

When launching Source app

- Please use the below public method when initiating the app or when app is in foreground : This will trigger offline syncing

```
Swift
ClearQuote.shared.initiateOfflineInspectionsSync()
```

- If supporting Bodystyle verification feature, add below code to refresh vehicle list in the background

```
Swift
if ClearQuote.shared.isCQSDKInitialized() {
    if ClearQuote.shared.bodystyleVerificationEnabled {
        ClearQuote.shared.refreshVehicles()
    }
}
```

Demo app repo: <https://github.com/clearquotetech/ClearQuote-iOS-SDK-Demo-app>

If you face issues when integrating this SDK inside your IOS app, please report details about the issue by providing detailed description along with screenshots, crash logs, stack traces, etc at this link

<https://github.com/clearquotetech/cq-ios-sdk/issues>

If you have queries then please contact - sharath@clearquote.io and akhila@clearquote.io